

Parallelizing GPT-2

By: Siggy Sigler, Sam Abderholden, Dawson Matthews

Introduction

Serial Implementation / Performance

name_list: [[BOS, E, L, L, A, BOS], [name_B], [name_C]]

BOS = Beginning of Sequence

Serial:

For name in name_list:

 For token in name:

 calculate_embedding(token)

 For each layer:

 calculate_attention(token)

 calculate_next_hidden_state(token)

 calc_loss(token)

 sum_loss(tokens)

Serial Analysis

Memory vs CPU Bound: TODO

Big-O

$O(\text{num_names} * \text{layers} * (C_{\text{attn}} + C_{\text{mlp}}))$

Static Analysis: TODO

Serial Implementation / Performance

name_list: [[BOS, E, L, L, A, BOS], [name_B], [name_C]]

BOS = Beginning of Sequence

Serial:

```
For name in name_list:
  For token in name:
    calculate_embedding(token)
    For each layer:
      calculate_attention(token)
      calculate_next_hidden_state(token)
    calc_loss(token)
sum_loss(tokens)
```

Parallel (per batch):

```
calculate_embeddings(name_list)
For each layer:
  calculate_attention_scores(name_list)
  calculate_next_hidden_state(name_list)
calc_loss(name_list)
```

Parallelization Strategy

- **Batching**
 - For each sequence launch a transformer -> run transformer on all sequences at once
- Store all of our information in a flattened array
 - Allows assignment of work easily by batch to threads
- Store a “workspace” on the GPU that holds weights, hidden states, etc.
- Large portions of the forward pass are parallelizable
 - **Embedding calculations**
 - **RMS normalization and linear calculations**
 - **Attention**
- **Layers must be performed sequentially**

Performance Evaluation

	Python Serial	C++ Serial	CUDA Parallel
Names: Layers: Embeddings:			
Names: Layers: Embeddings:			
Names: Layers: Embeddings:			